الجمهورية العربية السورية

جامعة دمشق ـ كلية الهندسة المعلوماتية

السنة الرابعة ـ قسم النظم والشبكات الحاسوبية ـ نظم تشغيل-2-

Operating system - 2 - final project

Project Title:

Deploying a Single Web Application with Docker Compose, Self-Signed SSL, Apache Load Balancer, Monitoring with Prometheus & Grafana, and AI-Powered Log Classification.

Objective:

Design and implement a multi-containers Docker environment that runs one web application with two instances, secured with self-signed SSL certificates, and load-balanced using Apache as a reverse proxy.

Additionally, integrate monitoring with Prometheus and Grafana to visualize system metrics such as request load, response times, and system health. AI-Powered Log Classification & Traffic Analysis**,** enabling intelligent detection of errors, security events, and DDoS-like traffic patterns through automated log inspection

Project Requirements:

1.Web Applications:

- Develop or use existing simple web apps.
- Containerize the web app using Docker.
- Deploy two instances of the same web app for load distribution.

2.SSL Configuration:

- Generate self-signed SSL certificates.
- Configure the web app containers to serve over HTTPS using these certificates.

3.Docker Compose Setup:

- Write a docker-compose.yml file that:
  - – Defines two instances of the same web application.
  - – Includes a container for Apache that acts as a load balancer.
  - – Sets up network configurations for service discovery.

4.Apache Load Balancer:

- Configure Apache as a reverse proxy with SSL termination.
- Load balance incoming HTTPS traffic across the two web app instances.
- Use self-signed certificates in Apache for SSL.
- Ensure Apache forwards requests correctly to backend instances.

5. Monitoring Setup:

- Integrate Prometheus to scrape metrics from the web app containers and Apache.
- Set up Grafana dashboards to visualize the metrics such as request load, response times, and system health.
- Configure monitoring in docker-compose.yml to include Prometheus and Grafana containers.

6. Testing:

Verify that:

- Both web app instances are running.
- HTTPS is functioning with self-signed certificates.
- Apache correctly load balances requests.
- The system handles multiple requests, demonstrating load distribution.
- Monitoring dashboards display real-time metrics and system health, containers status, consumption per container of cpu and memory.

7. AI-Powered Log Classification & DDoS-Like Traffic Analysis:

To enhance the observability and analytical capabilities of the system, an additional AI-assisted component must be integrated into the deployment.
This component introduces a lightweight AI Log Classifier running as a dedicated container inside Docker Compose.
It analyzes Apache logs in real time and classifies them into operational and security-related categories using rule-based logic (no training required).

7.1 Objectives

- Provide intelligent log analysis without machine-learning training.
- Detect:
  – Operational errors (500, exceptions)
  – Security alerts (401, 403, failed authentication)
  – Suspicious patterns (SQL injection, XSS indicators)
  – High-traffic anomalies (DDoS-like behavior, e.g., repeated 429 responses)
- Expose classification results as Prometheus metrics.
- Visualize AI-generated insights in Grafana dashboards.

7.2 System Description

- A new container is added to run the AI log classifier (Python-based).
- Apache logs are shared with the classifier through a Docker volume.
- Each log line is processed using rule-based classification to determine its category.
- The classifier exposes counters (errors, security alerts, suspicious requests, 429 events) through a Prometheus-compatible **/metrics** endpoint.
- Prometheus scrapes these metrics and Grafana visualizes them in real time.

7.3 DDoS-Like Traffic Simulation

- To validate the classifier's ability to detect high-traffic anomalies, a Traffic Generator container is added to the Docker Compose setup.
- This service generates a large number of concurrent HTTPS requests to the Apache load balancer within the internal Docker network.
  When Apache rate-limiting thresholds are exceeded, it returns 429 Too Many Requests, which are logged and detected by the AI classifier as DDoS-like behavior

7.4 Expected Outcomes

- Enhanced monitoring depth through AI-assisted log interpretation.
- Real-time detection of abnormal traffic patterns and potential security events.
- Grafana dashboards displaying:
  – Error counts
  – Security-related log entries
  – High-traffic (429) events
  – Suspicious request patterns
- A more realistic, production-oriented monitoring environment.

System architecture

[Create Web App Files]
↓
[Create Dockerfiles for Web App]
↓
[Build Docker Images for Web App]
↓
[Generate SSL Certificates]
↓
[Configure Apache]
↓
[Set Up Monitoring Configs (Prometheus & Grafana)]
↓
[Add AI Log Classifier & Traffic Generator]
↓
[Write docker-compose.yml]
↓
[Deploy Containers]
↓
[Access & Monitor Deployment]

NOTES:

- The number of group members: **(3-4)**
- A paper report including the steps for the solution with screenshots.
- The date of the interview: **14/6/2026**

BEST WISHES
ENG. MOHAMMAD MERIE
2026